

AGENT PASSPORT

Technical Builder Guide — v9

C:\>DIR Global · KYA Challenge Pathway · Lexis Labs · September 2026

PROPOSITION

Agent Passport lets a bank prove that a particular AI agent is acting within a current, human-approved payment mandate.

THE ONE THING THE DEMO MUST PROVE

A valid agent — issued by a regulated provider, carrying a customer mandate — can still be manipulated into requesting an invalid action. The mandate stops it. The system flags it. A human decides.

HOW TO READ THIS GUIDE

PART A (pages 2–12) Core build guide for Tim. Build everything here. This is the submission.

PART B (pages 13–16) Advanced section for Jean for additional optional bonus technical depth.

READ THIS FIRST — TIM'S PRIORITY LIST

v0.2.0 ALREADY BUILT: Ed25519 (ALL THREE VERIFY ✓) · Standards Review Assistant 6 steps ✓ · Anomaly Detection panel ✓ · Issuance Flow v4 panels ✓

BUILD NOW (P0): Confirm poisoned invoice is wired end-to-end: live LLM reads INV-9001-poisoned → R.3 DENY visible in terminal. (~2 hrs). This is the defining demo moment.

BUILD NEXT (P1): Grounds Declaration J-rules — J.1–J.8 bank-side verification + HOLD state + approved-sources field in passport. (~1 day). Section A5 GAP 5.

LABELS ONLY, NO CODE (P2): Standards badges in UI + About page copy (2 hrs). Map Alex's five-layer framework to what already exists — do not refactor the credential structure.

DO NOT BUILD: Three-artefact architectural refactor. Agent handoff chain (Part B). HMRC secondary scenario. Read the architecture notes for orientation only.

PART A — CORE BUILD GUIDE

Every item in Part A must be built. The submission is complete without Part B.

A1. Challenge Alignment

The problem in one real-world example:

JPMorgan Chase already applies relationship-context checks to Zelle payments: "For your protection, we don't let you pay someone you've indicated that you only know through social media." That is a real bank, today, trying to infer human intent from behavioural signals — because it cannot tell legitimate payments from scam-facilitated ones. The gap it cannot close: it still cannot distinguish a payment submitted by a human treasury manager from one submitted by an autonomous AI agent acting on their behalf. Agent Passport closes that gap.

We enter the KYA Challenge Pathway with one solution area:

Bridging the Delegation Gap

"How can an authority verify that an autonomous agent is acting strictly within its human owner's authorised scope?"

The closest illustrative prototype is the Cryptographic Delegation Verifier. We claim that prototype. We build a clear working demonstration of its core principle, with technical depth the PhD can extend.

What the evaluator must understand in three minutes

- PayRail is a payment institution operating a specific AI agent.
- Northgate's finance director signs a structured payment mandate for that agent.
- The regulator runs an AI-assisted evidence review — the Standards Review Assistant. A human makes the approval decision. This is the brief's "KY-A Standards Sandbox Copilot."
- The bank cryptographically verifies agent identity and mandate before releasing any money.
- A normal payment succeeds.
- The AI agent is deliberately manipulated into requesting payment to the wrong account.
- The payment is blocked — despite valid identity and acceptable amount. The beneficiary is not in the mandate.
- The violation is recorded. A human supervisor decides to suspend and revoke.

That is the complete, memorable story. Do not add scope beyond what makes it clearer.

A2. Standards Integration — The Brief's References Applied

Why this section matters:

The brief's reference list (page 4) names specific technical standards: NIST NCCoE 2026 (OAuth 2.0, OpenID Connect, SPIFFE, Zero Trust), W3C DIDs and Verifiable Credentials, FATF 2025. A judge who has read the brief will check whether these are visible in the demo — not just mentioned in an About page. This section maps every cited standard to a specific component and tells Tim what to label visibly in the UI.

Brief reference	Standard / tool	How we use it — label this in the UI
NIST NCCoE 2026 OAuth 2.0	<code>RFC 9396 Rich Authorization Requests</code>	The customer mandate IS an OAuth 2.0 RAR. Label Tab 1: "Payment Mandate (OAuth 2.0 RFC 9396 Rich Authorization Request)." The <code>authorization_details</code> field is the mandate scope. RFC 9396 was designed for exactly this pattern.
NIST NCCoE 2026 OpenID Connect	<code>JWT profile + OIDC claims conventions</code>	Provider and authority attestations use JWT format with OIDC-compatible claims (iss, sub, iat, exp). Label the JWT header in Tab 1: "JWT (OIDC-compatible)."
NIST NCCoE 2026 SPIFFE / SVID	<code>Ed25519 key binding (RFC 7800 cnf)</code>	Our Ed25519 key pair, bound to the agent via RFC 7800 cnf, provides equivalent cryptographic workload identity to SPIFFE SVID. Add tooltip: "Agent key binding provides workload-identity guarantees equivalent to SPIFFE SVID." Note: SPIFFE extension is a production roadmap item.
NIST NCCoE 2026 Zero Trust	<code>9-rule deny-by-default bank verification</code>	Every payment is denied by default; explicitly permitted only when all 9 rules pass. This IS a Zero Trust implementation. Label Tab 3: "Zero Trust Payment Verification (deny-by-default, R.1–R.9)."
W3C Verifiable Credentials	<code>JWT-VC profile (no JSON-LD wrapper)</code>	The passport envelope is a W3C Verifiable Credential, JWT profile. Label Tab 1: "Credential format: W3C VC (JWT profile, RFC 7519)." JSON-LD wrapper not required; JWT-VC is the W3C-endorsed lightweight variant.
W3C Decentralized Identifiers (DIDs)	<code>Ed25519 key usable as did:key method</code>	Our Ed25519 public keys are compatible with the did:key DID method. Add one About page line: "Keys compatible with W3C did:key resolution. Full DID document support is a production extension." Do not claim full DID support.
FATF 2025 AML/CFT AI risks	<code>Named-beneficiary mandate field + amount ceiling</code>	FATF 2025 flagged AI-enabled circumvention of AML/CFT identity controls. The mandate's named-beneficiary field and amount ceiling are direct AML controls. State this explicitly in About page and on R.3 in Tab 3.
Tiger Research 2026 KYA market map	<code>Agent Passport positioned re: Visa TAP, ERC-8004</code>	Tiger Research maps KYA to Visa TAP (payment rail agents) and ERC-8004 (smart contract delegation). Agent Passport is the regulated-entity equivalent. About page: "Addresses the same delegation-gap problem within the regulated payment institution framework."

Builder instruction:

Every row in the "How we use it" column describes a UI label or About page addition. These are not new builds — they are labels on existing components. This single section answers the judge question "you list all these references but where are they in the demo?" Add the labels and tooltips. It takes two hours.

A3. Regulatory Grounding — Who Is the Regulator?

This section covers the institutional framing, the v4 issuance flow, and the critical distinction between bank-level payment verification and regulator-level oversight.

PayRail represents a payment institution with responsibilities under the Payment Services Regulations 2017 (PSR 2017), which require it to maintain controls over who submits payment instructions on its behalf. Agent Passport demonstrates what an Agent Assurance function would look like within the existing PSR 2017 authorisation framework — a proposed extension that certifies which specific AI agents are authorised, under what controls, and within what mandates.

This is a design prototype, not a claimed regulatory approval. The “regulator” tab in the demo represents what an FCA Agent Assurance function could look like if the PSR 2017 framework were extended to cover AI agents. It is grounded in existing regulatory logic, not invented wholesale.

Label Tab 2 as: “Regulatory Assurance (proposed PSR/2017 Agent Assurance function).” Add one line to the intro text: “This demonstrates what agent certification within the PSR 2017 framework would look like.”

A3a. Issuance Flow v4 — How Passports Are Created (ADOPTED)

What changed and why it matters:

The previous issuance flow required the authority to review every individual customer mandate. This was architecturally wrong: it created a bottleneck, mixed provider certification with customer authorisation, and made the system unscalable. The v4 flow separates these concerns cleanly.

PHASE 1 — Provider registers the agent model (once):

PayRail submits: model documentation, benchmarks, training type, the agent’s Ed25519 public key, configuration hash, and insurance certificate. No customer data. The authority reviews the model itself — not any specific customer’s use of it.

PHASE 2 — Authority approves the model and sets policy ceilings (once per approval):

The authority runs the Standards Review Assistant, makes a human decision, and applies the authority signature. It sets policy ceilings: maximum delegation amount, permitted action types, and any supervisory conditions (e.g. key rotation policy required). All future customer mandates must stay within these ceilings. The authority does not see or approve individual customer mandates.

PHASE 3 — Customer writes and signs their own mandate (self-service, live immediately):

Northgate’s finance director writes the mandate directly: beneficiary account, amount ceiling, expiry date. They sign it via corporate banking login. It goes live at once — no authority review delay. The bank R.1–R.9 check confirms the mandate stays within the authority’s policy ceilings at every payment.

Why this is architecturally correct:

The scope-containment invariant holds automatically: authority sets the ceiling, customer sets their limit within it, bank checks both on every payment. No per-mandate authority review is needed because the bank check enforces the ceiling at execution time. The authority only re-engages if something goes wrong — an anomaly flag or a revocation request.

A3b. Critical Distinction — Bank Check vs. Regulator Oversight

THE BANK VERIFIES EVERY PAYMENT. THE REGULATOR DOES NOT SEE EACH TRANSACTION.

The bank runs R.1–R.9 on every payment instruction:

Every time the AI agent submits a payment, the bank runs all nine rules in sequence. Deny-by-default. Sub-10ms. No AI inference. No human in the loop. This happens on every single payment, regardless of size or type. This is the enforcement layer.

The regulator acts on patterns and exceptions — not individual payments:

The regulator approved the agent model at Phase 2. It does not receive each payment instruction in real time. It receives: AML pattern reports from the bank, escalation alerts when violation thresholds are triggered, and revocation requests when the system flags an agent. Individual payment routing is not regulated transaction-by-transaction at the regulator level.

Plain English:

The bank is the bouncer checking every ID at the door. The regulator is the licensing authority that certified the club is safe to operate. The bouncer does not call the licensing authority for every customer who walks in. But if a pattern of incidents emerges, the licensing authority steps in.

What this means for the demo:

Tim’s observation that large B2B transactions are not individually regulated at the regulator level is correct. But the bank still runs R.1–R.9 on every payment instruction the AI submits. These are different layers. If a judge asks “does the regulator approve each transaction?” the answer is: “No. The bank enforces the rules at execution time. The regulator certified the agent at assurance time and steps in on anomalies.”

A3c. B2B Bank Incentive — Honest Framing

What actually drives a B2B bank to adopt Agent Passport:

1. AML compliance (PSR 2017 applies regardless of B2B or B2C):

The Payment Services Regulations 2017 require banks to maintain controls over who submits instructions on behalf of customers. As AI agents begin submitting large B2B payments, a bank with no mechanism to distinguish an authorised agent from an unauthorised one has a compliance gap. Agent Passport closes that gap with a signed, auditable record of what was checked and why the payment was allowed.

2. Market access gatekeeping (commercial moat):

A bank that adopts Agent Passport becomes a certification point in the AI agent payment ecosystem. Payment institutions whose agents carry a valid passport can access the bank’s rails. Those without cannot. The bank controls the gate. This is a commercial position, not just a compliance requirement.

3. Risk management (liability without a verification record):

A B2B bank that processes a large payment submitted by an AI agent, with no mechanism to verify the agent was authorised, has accepted operational risk with no audit trail. If the payment turns out to be outside the customer’s intended scope, the bank has no signed record of what it checked. Agent Passport gives the bank that record.

NOTE — APP fraud mandatory reimbursement is primarily a consumer and SME story:

The 2024 APP fraud mandatory reimbursement regime applies to consumer and small business payments. For large B2B transactions, the mandatory reimbursement framework does not apply in the same way. In the B2B demo context, lead with AML compliance and market access. Keep the APP fraud framing for consumer payment system context — it belongs in A9 as supporting background, not as the primary B2B commercial driver.

A4. Current Prototype Status

Assessed against v5 scope. Be honest with judges about what is live vs. demonstrated.

Provider → Regulator → Customer Panels (Issuance Flow v4)	✓ DONE	All three panels operational in v0.2.0. Ed25519 cryptographically live: ALL THREE VERIFY confirmed. Standards badges in UI. Customer mandate signed (Helen Marsh, three payees). Three-signature envelope complete.
Regulator Panel	✓ DONE	Standards Review Assistant fully operational: 6-step AI-assisted review live in v0.2.0. Anomaly Detection and Escalation panel live: violations table, threshold alert, Suspend/Revoke controls. A.1–A.8 automated checks running.
Action Terminal	✓ DONE	R.1–R.9 deny-by-default live. Eight proposed instructions with correct ALLOW/DENY/ESCALATE outcomes. Ed25519 verification real (not field-matching). Poisoned invoice button (INV-9001-poisoned) present. CONFIRM: run it and verify R.3 DENY appears in terminal output.
Audit Trail	✓ DONE	Hash-chained audit log (sha256(prev_hash + entry)), 14 entries, INTACT in v0.2.0. Vouch Rail active. CONFIRM: poisoned invoice run appends a violation entry with correct hash chain.
Grounds Declaration (J- rules)	✗ MISSING	J.1–J.8 bank-side source-integrity checks not yet built. HOLD state not wired. Approved-sources field not in passport. Sole remaining build gap. Build only after poisoned invoice confirmed end-to-end. See Section A5 GAP 5.
Standards Badges + About Page	✓ DONE	OAuth 2.0 RFC 9396 W3C VC JWT RFC 7800 cnf Ed25519 RFC 8037 badges visible throughout UI in v0.2.0. About page copy from Section A9 can be pasted in — low priority, do after P0 confirmed.

A5. Gap Analysis — What to Build

Four gaps. Three require code. One (Gap 4) requires only labels and copy.

GAP 1 — Poisoned Invoice Demo [P0 — CONFIRM WIRING]

WHY:

"Detect autonomous exploitation of digital public infrastructure" is in the top-level brief. Without a live failure scenario the demo only shows the happy path. The moment the AI is manipulated and the mandate stops it is the single most important and most memorable moment of the demo.

HOW:

Scenario:

Invoice INV-9001 from "Fenwick Timber Ltd". Real account: 10101010. Attacker replaces PDF with poisoned version showing account 99887766. AP Agent reads the poisoned invoice via live LLM call. Produces payment instruction to 99887766. Submits to bank.

Bank denial (Rule R.3 — beneficiary mandate check):

```
Mandate: { beneficiary: "10101010" (Fenwick Timber) }
Request: { beneficiary: "99887766" } ← attacker account
Result: DENY. R.3 failed. FATF-aligned AML control triggered.
```

On screen (four things in sequence):

1. Extracted invoice text showing the poisoned account number.
2. AI’s payment instruction with the wrong account highlighted red.
3. Bank denial panel: R.3 highlighted red, FATF note visible.
4. Caption: "The AI read a manipulated invoice and would have paid the wrong account. The mandate stopped it."

Immediately after denial:

Violation record created in Tab 2 exception panel. Status: OPEN.

WHAT THE MANDATE DOES AND DOES NOT PREVENT — be honest about scope:

The mandate enforces: beneficiary account number (R.3), amount ceiling (R.5), passport validity status (R.1–R.2), and expiry (R.4). The demo shows beneficiary substitution — this is the primary AML control. The mandate does not enforce invoice text accuracy or supplier name matching; an attacker who changes the amount but stays under the ceiling, or changes the supplier name but keeps the correct account number, would not be stopped by this check alone. These are acknowledged limitations. If a judge raises this, the correct answer is: "The mandate is an AML control on where money goes and under whose authority — not an invoice verification engine. Full invoice semantic validation is a production extension." Do not overclaim.

Vouch.finance (Tab 5) — add one-line explanation:

Tab 5 mirrors the mandate to vouch.finance as an "AI Voucher." This demonstrates open-ecosystem interoperability: the same mandate that a regulated bank verifies can also be anchored in an open standard, addressing the Tiger Research KYA market map’s open/DeFi dimension. Add this sentence to the Tab 5 header: "Mandate anchored as AI Voucher on vouch.finance — demonstrating interoperability with the open agent-authorisation ecosystem."

GAP 2 — Standards Review Assistant [✓ DONE IN v0.2.0]

WHY:

The brief specifically cites a "KY-A Standards Sandbox Copilot" as an illustrative prototype. The Standards Review Assistant is exactly that: AI that reads a credential bundle, maps it to the relevant rule pack, surfaces compliance gaps, generates test cases, and assists a human regulatory decision-maker. A static evidence form with a human rubber-stamping it is a credential system. The Standards Review Assistant is an AI-assisted regulatory copilot — it is the brief's own language brought to life.

HOW — six steps in sequence:**Step 1 EVIDENCE READ**

Parse the three passport components. Display a summary card: what has PayRail claimed? What has Northgate authorised? What does the authority certify?

Step 2 STANDARDS RULE MAP

Load the versioned rule pack (payments-2026.09). Map each requirement to the evidence. Flag any requirement not covered. Show the mapping visibly — this is the "Standards Sandbox Copilot" element.

Step 3 TEST CASE GENERATION

Generate 3–5 test cases specific to this passport: "Payment to non-mandated supplier", "Amount £1 above limit", "Expired passport", "Key rotation test". Show as visible test cards. These are generated from the rule pack gaps — not open-ended LLM brainstorming.

Step 4 SANDBOX RUN

Submit each test to the bank verification endpoint (Tab 3 logic reused). Record PASS or FAIL per test. Display against each card.

Step 5 RECOMMENDATION

Structured output: APPROVE / APPROVE WITH CONDITIONS / REFER. Include reasoning. Label: "AI-assisted recommendation — human decision required."

Step 6 HUMAN SIGN-OFF

Human regulator reviews recommendation and approves or refers. Only a human click activates the passport. Authority signature is applied on approval.

CRITICAL — Cryptographic verification must be live, not simulated:

The bank endpoint (Tab 3) must actually verify the Ed25519 signature on the passport — not just check JSON field values. Field-matching is not cryptographic verification. The check must call the signature verify function (e.g., `nacl.sign.open()` or equivalent) on the authority and provider signature bytes, using their public keys. If a judge asks "show me the signature verification" and the code does a string comparison, the cryptographic claim collapses entirely. This is a one-hour fix and the single most important technical credibility point in the whole submission.

GAP 3 — Exception Panel + Pattern Detection + Revocation [✓ DONE IN v0.2.0]**WHY:**

Blocking one attack is prevention. The brief requires detection. Detection means recognising a pattern across events and escalating to a human supervisor. The exception panel records the event — but adding a simple threshold rule ("two R.3 failures within 24 hours triggers automatic supervisor alert") changes the framing from prevention to detection. This is one extra condition, not a new build.

HOW:

Exception panel (violations table):

Columns: Timestamp | Agent ID | Rule violated | Payment attempted | Outcome | Status (OPEN / INVESTIGATING / RESOLVED). The malicious invoice denial creates a row. Label the panel: "Anomaly Detection & Escalation."

Pattern threshold (the detection element):

Add a configurable rule: ≥ 2 violations of the same type within 24 hours triggers an automatic supervisor alert banner at the top of Tab 2. This is one if-condition on the violation log. It changes the narrative from "we blocked an attack" to "we detected a pattern."

Revocation flow (three states):

SUSPENDED: payments blocked at R.2. INVESTIGATING: supervisor reviews exception analysis. DECISION: REVOKE (voucher deleted, passport REVOKED) or REINSTATE (false positive).

GAP 4 — Standards Labels [✓ DONE IN v0.2.0]

WHY:

A judge who has read the brief will check whether the referenced standards are visible in the demo. They are already implemented — they just need to be labelled. This is the highest-leverage two hours in the build. Section A9 has all the copy to paste.

HOW — three UI additions and About page copy:

Tab 1: Add standards badge row beneath the three signature panels. Four badges: "OAuth 2.0 RFC 9396" | "W3C VC JWT" | "RFC 7800 cnf" | "Ed25519 RFC 8037".

Tab 2: Add label: "Regulatory Assurance (proposed PSR/2017 Agent Assurance function — design prototype)."

Tab 3: Add "Zero Trust — deny by default" label in the header. Add FATF note on R.3: "R.3: Named-beneficiary mandate check (FATF 2025 AML/CFT alignment)."

About page: Paste the complete copy from Section A9.

A5. GAP 5 — Grounds Declaration + J-Rule Verification

[P1 — only after ALL P0 items complete. Source: Jean's Grounds Declaration document, 6 Sep 2026.]

The existing R-rules check the envelope of a payment: right passport, right beneficiary, right amount. They cannot check what the agent acted on. An agent manipulated via a chat message still produces a perfectly clean payment — correct account, valid passport, amount under ceiling, every R-rule green. This layer catches it by checking whether the agent drew on sources its passport permits.

The core idea in one paragraph:

The model is the part that thinks — it reads text and writes text, it cannot open files or move money. The harness is ordinary software wrapped around the model — it does the fetching, and it can record every source it fetched: what kind of source, which reference, when. That record is written by deterministic software, signed with a key registered at issuance, and never seen or written by the model. So instead of asking the agent to explain itself and trying to judge whether the explanation is honest — which we cannot verify — we check what it actually touched. That we can verify mechanically, every time.

J-rules — ESCALATE only (never DENY):

J.1 Source not on passport's approved list (e.g. EXTERNAL_MESSAGE not permitted for a payments agent).

- J.2 Invoice reference cited by model does not exist in the ledger.
- J.3 Invoice already settled (duplicate payment attempt).
- J.4 Amount submitted does not match the invoice amount.
- J.5 No supporting document present at all.
- J.7 Grounds record missing or signature invalid.
- J.8 Instruction absent or from an unrecognised source (instruction not from a named principal or registered scheduled job).

Result when any J-rule fails: HOLD — named human confirms.

The payment is never blocked by a J-rule alone. J-rules escalate; R-rules deny. This is the legally correct design: you cannot auto-block a payment because the agent consulted a supplier message — you escalate to a named human.

WHAT TIM ACTUALLY BUILDS (green) vs what is fixture data (grey):

BUILD: Bank-side check_grounds() function

Receives the payment request JSON (which includes the grounds array + runtime attestation). Calls verify_attestation() against the runtime_key in the passport. Loops through grounds entries, checks each channel against approved_input_channels in the passport. Checks each cited reference against the ledger (J.2–J.5). Returns list of (rule_code, reason) flags.

BUILD: HOLD state in the bank UI

When any J-rule fires, the payment status becomes HOLD (not DENIED). An exception row appears in the violations table. A named approver is notified. The payment cannot proceed until the approver confirms.

BUILD: approved_input_channels field in the passport JSON

Add a field to the passport structure: approved_input_channels: ["TASK_BRIEF", "SUPPLIER_PORTAL_INVOICE"]. The bank reads this field during check_grounds().

FIXTURE DATA (not operator harness instrumentation):

The code that wraps the agent's tool calls and produces the grounds record belongs to the operator, not to us. For the demo, supply the grounds record as scripted fixture data — a JSON payload showing what an instrumented runtime would have produced. Build the bank-side verification against that fixture. Jean's Part B Section B1 says this explicitly.

Honest limitation — what the bank can and cannot prove:

The bank can verify: (1) the log was not altered after signing — the attestation signature on the grounds record is valid. (2) Each channel listed was on the approved list at the time of submission.

The bank CANNOT verify: that the log is COMPLETE — the operator's runtime might have fetched additional sources that did not appear in the grounds record. Completeness is a legal and contractual claim (passport condition C1 requires the operator to instrument all sources), not a purely technical one. This is an honest limitation. State it if asked.

Two-case proof — why both R-rules and J-rules are needed:

Case 5 (external message → Stage 7b): Same supplier. Same account. Amount under ceiling. All nine R-rules: green. J.1: HOLD — acted on EXTERNAL_MESSAGE, not on the approved-sources list. This proves J-rules are non-redundant: R-rules would have passed this payment.

Case 10 (poisoned invoice → Stage 7): J-rules all pass (invoice arrived through approved supplier portal). R.3 fails (wrong account). DENY. This proves the two layers cover different failures, not the same one.

A judge will ask: "Why do you need both layers?" This is the answer.

A6. Build Priority

	Item	What to do
DONE	Ed25519 Crypto Fix	ALL THREE VERIFY shown in v0.2.0. nacl.sign.open() live. No action needed.
DONE	Issuance Flow v4	Provider Panel + Regulator Panel + Customer Panel all operational. Three-panel flow live. No action needed.
P0	Poisoned Invoice End-to-End	Button exists (INV-9001-poisoned). CONFIRM: run it and verify live LLM extracts account 99887766 → R.3 DENY appears in terminal → violation appended to audit trail. ~2 hrs. This is the defining demo moment.
DONE	Standards Review Assistant	6-step AI-assisted review operational in Regulator Panel (v0.2.0). No action needed.
DONE	Anomaly Detection + Revocation	Violations table, threshold alert, Suspend/Revoke controls live in Regulator Panel. No action needed.
DONE	Standards Badges	OAuth 2.0 RFC 9396 W3C VC JWT RFC 7800 cnf Ed25519 RFC 8037 visible in UI. No action needed.
P1	Grounds Declaration J-rules	J.1–J.8 bank-side verification + HOLD state + approved-sources field in passport. ~1 day. Build ONLY after P0 confirmed. See Section A5 GAP 5.
P2	Standards Labels + About Page	UI badge row in Tab 1, labels in Tab 2/3, three-artefact naming, paste About copy from A9. ~2 hours.
OPT	Agent Handoff Chain (Part B)	Orchestrator → Execution Agent + verify_chain. Only after P0 + P1 complete and tested end-to-end. See Part B.
CUT	Alex Three-Artefact Refactor	Map Alex's Passport / Principal Mandate / Action Intent Manifest to labels on what already exists. Do not refactor the credential structure. Not a build task — a labelling task (P2).
CUT	HMRC Secondary Scenario	Do not build. Mention as future extension only.

Order matters:

P0 items first, in order. The crypto verification fix takes one hour and should be done before any other work. Do not touch Part B until P0 and P1 are working end-to-end through the full demo flow. The P2 standards labels take two hours and should be done alongside the P1 build, not left to the end.

A7. Complete Demo Design — 8 Stages

This is the full demo architecture. Walk this sequence in practice. Section A7b gives the condensed presentation script that fits the hackathon time constraint.

Stage 1 ✓ CURRENT	Standards Review Assistant (Regulator Panel) Standards Review Assistant is LIVE in v0.2.0. Open Regulator Panel. Show 6-step AI-assisted review. Point out: "This is the KY-A Standards Sandbox Copilot the brief describes." Human sign-off. Authority signature applied. NOTHING TO BUILD here. Brief ref: <i>Standards Sandbox Copilot (illustrative prototype in brief).</i>
Stage 2 ✓ CURRENT	Passport Issuance (Tab 1) Show Tab 1. Point to standards badge row. Three-signature envelope (Authority / Provider / Customer). Standards badges: OAuth 2.0 RFC 9396 W3C VC JWT RFC 7800 cnf Ed25519 RFC 8037. Status: ACTIVE. Brief ref: <i>W3C VCs, RFC 9396, NIST NCCoE OAuth 2.0 and OpenID Connect.</i>
Stage 3 ✓ CURRENT	Normal Payment — Happy Path (Tab 4) Clean invoice: Fenwick Timber, account 10101010, £2,500. AP Agent submits. Bank runs Zero Trust check (R.1–R.9, deny-by-default). All green. ALLOW. Signed receipt. Brief ref: <i>NIST NCCoE Zero Trust principle.</i>
Stage 4 ✓ CURRENT	Mandate Boundary Test (Tab 3) Change amount to £12,000 (over £10,000 limit). Submit. R.5 (amount ceiling) fails. DENY. Signed denial receipt. Brief ref: <i>Shows quantitative scope containment.</i>
Stage 5 ✓ CURRENT	Vouch Rail Mirror (Tab 5) Show Tab 5. Mandate mirrored as AI Voucher on vouch.finance sandbox. Status ACTIVE. One sentence: "Same mandate verifiable in the open agent-authorisation ecosystem — Tiger Research KYA alignment." Brief ref: <i>Open interoperability. Tiger Research KYA market map.</i>
Stage 6 ✓ CURRENT	PAUSE — set up the adversarial moment Say to judges: "The agent has a valid identity. PayRail is certified. The amount we're about to send is within the limit. Watch what happens when the AI is deceived." No action. This is the verbal setup. Brief ref: <i>Framing moment. Do not skip.</i>
Stage 7 ⚠ CONFIRM	Poisoned Invoice — AI Fails, System Contains It (Action Terminal) Button INV-9001-poisoned EXISTS in Action Terminal (v0.2.0). BEFORE DEMO: run it and confirm end-to-end: live LLM reads poisoned PDF → extracts 99887766 → R.3 DENY shown in terminal → violation entry in audit trail. Once confirmed this stage is CURRENT. Caption for judges: "The AI read a manipulated invoice and would have paid the wrong account. The mandate stopped it."

	Brief ref: <i>Top-level brief: "detect autonomous exploitation of DPI." FATF 2025 AML/CFT alignment.</i>
Stage 7b → BUILD (only if Grounds Declaration is complete)	J-Rule Demo — External Message (Tab 4) Same supplier. Same account (10101010). £2,400 — well under ceiling. Load a scenario where a message arrived asking the agent to pay early. Agent submits. All nine R-rules: green. J.1: HOLD — acted on EXTERNAL_MESSAGE, not on the approved-sources list. Exception row created. Named approver notified. Say: the payment itself is fine. What is wrong is where the instruction came from. The bank can see that, because the harness recorded what the agent drew on — and the passport says which sources this agent is allowed to act on. Brief ref: <i>Proves J-rules cover a different failure mode from R-rules. Two-case proof: Case 5 (J-rule catches external message) vs Case 10 (R.3 catches wrong account).</i>
Stage 8 ✓ CURRENT	Escalation and Revocation (Regulator Panel) Anomaly Detection and Escalation panel is LIVE in Regulator Panel (v0.2.0). Violations table, threshold controls, Suspend/Revoke buttons present. CONFIRM: after poisoned invoice run, trigger Revoke and verify R.2 blocks a subsequent payment. Audit trail hash-chained (14 entries, INTACT). NOTHING TO BUILD here. Brief ref: <i>Accountability loop: detection → human decision → enforcement → signed audit record.</i>

A7b. 3-Minute Judging Presentation Script

Why this section is separate:

The full 8-stage demo is how the system works. The 3-minute presentation is how you show it to judges. These are different. You do not show all 8 stages live — you show 5 moments and name the others. The goal is to hit the brief's four criteria (identity, authority, detection, escalation) and leave one unforgettable moment.

Time	Stage	Tab / Screen	What to say / show
0:00–0:25	Problem & Setup	Verbal only	Say: "Banks today cannot distinguish a payment from a human treasury manager from one submitted by an autonomous AI agent. JPMorgan Chase is already adding friction to payments — but even their check doesn't solve this." Open Tab 1.
0:25–0:50	Passport & Standards	Tab 1	Point to the three-signature envelope and the standards badge row. "This is a W3C Verifiable Credential carrying an OAuth 2.0 Rich Authorization Request. The agent's identity is cryptographically bound via RFC 7800. The regulator approved this using the Standards Review Assistant — the KY-A Standards Sandbox Copilot the brief describes."
0:50–1:05	Happy Path	Tab 4	Send clean invoice. Bank runs Zero Trust check (R.1–R.9, deny-by-default). ALLOW. "Normal payment works." Move fast.
1:05–1:15	PAUSE	Verbal only	Look up. "The agent is valid. The provider is certified. The amount is within limit. Watch what happens when the AI is deceived."
1:15–2:00	THE MOMENT	Tab 4 → Tab 3	Load poisoned invoice INV-9001. Agent reads it live. Extracts wrong account. Bank: R.3 DENY. FATF note visible. Caption on screen. Let it land. 10 seconds of silence here is fine.
2:00–2:25	Detection & Escalation	Tab 2	Show violation in exception panel. "The system recorded this. Two of these in 24 hours triggers an automatic supervisor alert — that's detection, not just prevention." Show REVOKE. Passport → REVOKED. Voucher deleted.
2:25–2:50	Standards & Audit	Tab 2 / About	Flash the Standards Review Assistant. "This is what regulated AI assurance looks like — AI-assisted review, human decision at every step. The audit trail is signed and complete."
2:50–3:00	Close	Verbal	One sentence: "Today a bank cannot tell an AI from a human. Agent Passport gives it the framework to do so — cryptographically, deterministically, sub-10ms per payment."

Presentation rules:

Practice the PAUSE (Stage 6) until it feels natural. It is the most important 10 seconds. Do not rush past the DENY result — let it render on screen before speaking. The Standards Review Assistant IS live in v0.2.0 —

show it briefly, name it as the KY-A Standards Sandbox Copilot, then move on. Judges will probe it in Q&A. Confirm the poisoned invoice end-to-end before the presentation.

A8. Anticipate These Questions — Know the Answers

These are the questions a regulatory or DPI evaluator will ask. Practise them before the demo.

Who signed the mandate, and how do you know they were authorised to?

Northgate's finance director signed it via corporate banking login — the same authorisation a CHAPS payment instruction requires today. In the prototype this is simulated; in production it is the organisation's existing payment-authorisation workflow. The customer mandate is the only component the customer signs. PayRail and the regulator cannot alter it.

Why does the regulator need this? Can't the bank already do this?

The bank already checks WHAT — the beneficiary, amount, reference. It cannot currently check WHO SUBMITTED IT: which specific AI agent, under which delegated authority, certified by which regulatory function. The 2024 APP fraud liability reform gave UK banks a direct incentive to make this distinction — but no technical framework exists to do so. Agent Passport is that framework.

What does the mandate not protect against?

The mandate enforces where money goes (beneficiary account) and how much (ceiling). It does not enforce invoice text accuracy or supplier name matching. An attacker who substitutes the account number is stopped. An attacker who manipulates the amount but stays under the ceiling is not stopped by this mechanism alone. We say this clearly: the mandate is an AML control on money destination and authority, not an invoice verification engine. Full semantic validation is a production extension.

What happens if the agent's key is stolen?

Revocation. The regulator or Northgate triggers revocation through the system. Passport moves to REVOKED. All subsequent payments denied at R.2 (passport status check). The vouch.finance voucher is deleted. Key rotation is one of the conditions the Standards Review Assistant flags at assurance time.

Is the cryptography real or mocked?

Real. The bank endpoint calls the Ed25519 signature verify function against the authority and provider public keys. It does not match fields — it verifies the signature bytes. If you changed a single byte of the payload the verification would fail. That is not a UI label; that is a cryptographic guarantee.

What data does the regulator receive? What about privacy?

Violation records: rule breached, outcome, timestamp, agent ID. Not payment amounts or supplier names unless an exception is escalated and the regulator requests full context. The design is privacy-preserving by default — the regulator sees patterns, not transaction content.

Can this operate at Faster Payments or CHAPS volume?

Yes. The bank-side verification is a deterministic rule check with no AI inference at execution time — sub-10ms per payment, horizontally scalable, architecturally identical to existing fraud rule engines. The AI runs only at assurance time (Tab 2, once per passport) and at the agent's own LLM inference (Tab 4). Neither is on the critical payment path.

How is this different from a standard payment mandate?

A standard mandate controls the payment — beneficiary, amount, reference. Agent Passport adds a layer above it: it controls the agent submitting the payment — which specific AI agent, with which delegated authority, verified by which regulatory function. Today a bank cannot distinguish a payment from a human treasury manager from one by an autonomous AI agent. Agent Passport creates that distinction.

Does the regulator approve each payment the AI submits?

No. The regulator approved the agent model at assurance time (Phase 2 of the issuance flow). It does not see individual payment instructions in real time. The bank runs R.1–R.9 on every payment the AI submits — deny-by-default, sub-10ms. The regulator acts on patterns: it receives AML reports, escalation alerts when the violation threshold is hit, and revocation requests from the bank. If a judge asks “does the regulator approve each transaction?” the answer is: the bank enforces at execution; the regulator certified the agent at assurance and steps in on anomalies. These are different layers.

Why would a large B2B bank adopt this?

Three reasons. First, AML compliance: PSR 2017 requires banks to maintain controls over who submits payment instructions on behalf of customers. AI agents submitting large payments without a verification mechanism create a compliance gap. Agent Passport closes it with a signed audit record. Second, market access: a bank that adopts Agent Passport controls which AI agents can use its payment rails — a commercial gate position. Third, risk management: processing a payment from an AI agent with no verification record means accepting operational risk with no paper trail. Note: the 2024 APP fraud mandatory reimbursement regime applies primarily to consumer and SME payments; for large B2B, the driving incentive is AML compliance and market access, not mandatory reimbursement.

If the grounds log can be faked by the operator, what does it actually prove?

Three things. First: the log was not altered after signing — the attestation signature is valid (bank verifies this mechanically). Second: if the log is incomplete (the operator omitted sources), the operator has breached passport condition C1, which is a contractual and regulatory obligation. The passport can be revoked and the breach is documented. Third: on any incident, the bank can prove a gap between what the grounds record shows and what the payment outcome was. That gap is evidence of operator misconduct, not of a system that lacks controls. The limitation is real — completeness is a contractual claim, not a purely technical one — but that is the correct framing.

How does your architecture relate to Alex's five-layer framework?

Alex's framework maps cleanly to what already exists. Layer A (agent assurance) = authority signature on the passport. Layer B (principal mandate) = customer mandate (RFC 9396-style). Layer C (transaction intent) = payment instruction signed by the agent's Ed25519 key. Layer D (runtime policy decision) = the bank's R.1–R.9 check + J.1–J.8 check. Layer E (regulatory supervision) = AML pattern reports + escalation alerts to the regulator. Nothing needs to be rebuilt. Alex's three-artefact model (Passport / Principal Mandate / Action Intent Manifest) maps to the three-signature credential envelope — label what already exists, do not refactor.

Why can't the operator just prevent this by disabling chat or blocking external messages?

They can and should — operator guardrails are the primary control. Concede that immediately. Then three things: First, a promise is not a verifiable fact. The bank cannot read the operator's code, cannot see which tools a developer wired up under deadline pressure, and cannot audit a system it does not own. Self-imposed controls are not verifiable controls. Second, the party bearing the loss is not the party running the agent. Under the 2024 APP fraud reform, the sending bank pays when a payment goes wrong — yet it has no visibility into the operator's systems. 'Trust our guardrails' is not acceptable to whoever writes the cheque. Third, guardrails drift; passports are pinned. The operator's guardrail is correct on day one. Six months later someone adds an email tool for a legitimate reason and nobody re-approved anything. With the passport, the allowed-sources list is fixed at issuance and signed — widening it means going back to the regulator, and the bank sees the new

version. Final point: if the operator does its job perfectly, this layer catches nothing. That is true of every assurance control ever built. What it adds is a signed, checkable record that the control held — which is what makes negligence provable afterwards.

A9. About Page — Complete Paste-in Copy

Copy verbatim. Do not embellish beyond what the demo shows. Do not claim more than what is built.

What it is

Agent Passport focuses on bridging the delegation gap. It enables a payment institution's AI agent to carry a verifiable, human-approved payment mandate, and enables the relying bank to verify every payment action against that mandate before execution. Agent identity, regulatory assurance, action-time enforcement, anomaly detection, escalation and revocation are supporting components required to make that delegation control credible.

What it is not

This prototype does not attempt to solve: biometric synthetic-identity detection; agent swarm detection at infrastructure scale; complete cross-border mutual recognition; legal liability calculation or consumer-redress compensation; or systemic payment-infrastructure risk modelling. The mandate enforces money destination and authority — not invoice semantic accuracy; full invoice verification is a production extension. These are acknowledged problems in the broader KYA landscape, noted as production roadmap items.

Why it matters now

Banks today cannot distinguish a payment instruction submitted by a human treasury manager from one submitted by an autonomous AI agent acting on their behalf. Agent Passport is the proposed framework that closes this gap — cryptographically, deterministically, sub-10ms per payment.

For large B2B banks, the primary drivers for adoption are AML compliance (the Payment Services Regulations 2017 require controls over who submits payment instructions, regardless of whether the payer is a consumer or a corporation) and market access gatekeeping (a bank that certifies which AI agents can use its rails controls a commercial gate in the emerging agent-payment ecosystem). The 2024 APP fraud mandatory reimbursement reform is the consumer and SME incentive story; for large B2B, AML compliance and market access are the accurate framing.

Important distinction: the bank runs R.1–R.9 on every payment instruction the AI submits — deny-by-default, no AI inference, no human in the loop at execution time. When Grounds Declaration is implemented, the bank additionally runs J.1–J.8, escalating to a named human on any source-integrity concern. The regulator acts on patterns and anomalies, not on individual payments.

The FATF 2025 guidance on AI-enabled circumvention of AML/CFT identity controls makes this framework timely. The mandate's named-beneficiary field and amount ceiling are direct AML controls that prevent the AI from redirecting money, even when the AI itself has been manipulated. JPMorgan Chase already adds friction to detect intent signals on payments; Agent Passport makes that determination cryptographically provable, not behavioural.

Technical standards

Customer mandate: OAuth 2.0 Rich Authorization Request (RFC 9396), directly implementing NIST NCCoE 2026 guidance on OAuth 2.0 for agentic systems. Credential format: W3C Verifiable Credentials JWT profile (RFC 7519). Agent key binding: RFC 7800 cnf (equivalent workload-identity guarantee to SPIFFE SVID; SPIFFE extension is a production roadmap item). Signatures: Ed25519 / EdDSA (RFC 8037). Bank-side verification: deny-by-default Zero Trust architecture (NIST NCCoE 2026), deterministic, no AI inference at execution time. Ed25519 keys are compatible with W3C did:key DID resolution; full DID document support is a production extension. Revocation anchored via vouch.finance AI Voucher standard. Agent Passport addresses

the same delegation-gap problem as Visa TAP and ERC-8004, within the regulated payment institution framework.

Regulatory context

The regulator tab demonstrates what an Agent Assurance function would look like within the Payment Services Regulations 2017 framework. This is a design prototype of a proposed extension to an existing authorisation process — not a claimed regulatory approval. The assurance function is AI-assisted but human-decided at every consequential point. The Standards Review Assistant is the brief's "KY-A Standards Sandbox Copilot" brought to life.

PART B — ADVANCED TECHNICAL

For Jean. With it, the solution may win more convincingly on technical depth.

What Part B adds:

One agent handoff (Orchestrator → Execution Agent) with a formal `verify_chain` function that makes the scope-containment invariant programmatically demonstrable. This directly claims the brief's "Cryptographic Delegation Verifier" prototype language. The function runs visibly in Tab 3 during Stage 4 of the demo.

B1. The Core Safety Invariant

The mathematical claim the demo makes:

Delegation may narrow authority. It can never expand authority.

If the human grants scope S_0 , and successive agents receive scopes $S_1, \dots, S_n$, every valid delegation must satisfy:

$$S_n \subseteq S_{n-1} \subseteq \dots \subseteq S_1 \subseteq S_0$$

A payment action a is permitted only when $a \in S_n$ and all signatures, expiries, revocation and configuration checks pass.

B2. The One Agent Handoff

Add exactly one handoff. No more.

Chain:

```
Northgate Finance Director
  ↓ signs root mandate { beneficiary:"10101010", max_gbp:10000 }
AP Orchestrator Agent (rename existing AP Agent)
  ↓ reads invoice, delegates to Execution Agent:
    { beneficiary:"10101010", max_gbp:4000, sig:<orchestrator> }
Payment Execution Agent (new lightweight function)
  ↓ submits payment + delegation chain to bank
Bank: verify_chain checks  $S_1 \subseteq S_0$  and action  $\in S_1$ 
ALLOW / HOLD / DENY
```

For the malicious invoice (Stage 7 with chain)

Orchestrator reads poisoned invoice → produces delegation token with beneficiary "99887766" → Execution Agent submits → `verify_chain` fails: "99887766" $\notin$ mandate scope {10101010} → DENY. Tab 3 shows the chain failure step by step: (a) mandate scope ✓ (b) delegation $\subseteq$ mandate ✓ (c) action $\in$ delegation X.

B3. Scope Containment — Concrete Examples

Valid chain:

```
Human mandate:    { beneficiary:"10101010", max_gbp:10000, purpose:"timber" }
Agent delegation: { beneficiary:"10101010", max_gbp: 4000, purpose:"timber" }
Payment:          { beneficiary:"10101010", amount: 2500, purpose:"timber" }
Result: VALID. Every value  $\leq$  parent scope.
```

Invalid chain (malicious invoice):

```
Human mandate:    { beneficiary:"10101010", max_gbp:10000 }
Payment:          { beneficiary:"99887766", amount: 2500 }
Result: DENIED. 99887766  $\notin$  {10101010}.
```

B4. What to Say in the Pitch (if Part B is incorporated)

"Agent Passport enforces a single mathematical property: delegation may narrow authority, but it can never expand it. The human signs a root mandate. The bank verifies that every agent in the chain received a scope equal to or narrower than the one above it, and that the specific payment falls within the narrowest delegated scope. A manipulated agent cannot acquire powers the human never granted — even if it claims them."

Time estimate for Part B:

verify_chain function + Orchestrator/Execution Agent split: ~1 day. Tab 3 display update: ~2 hours. Total: ~1.5 days after Part A is complete and tested.